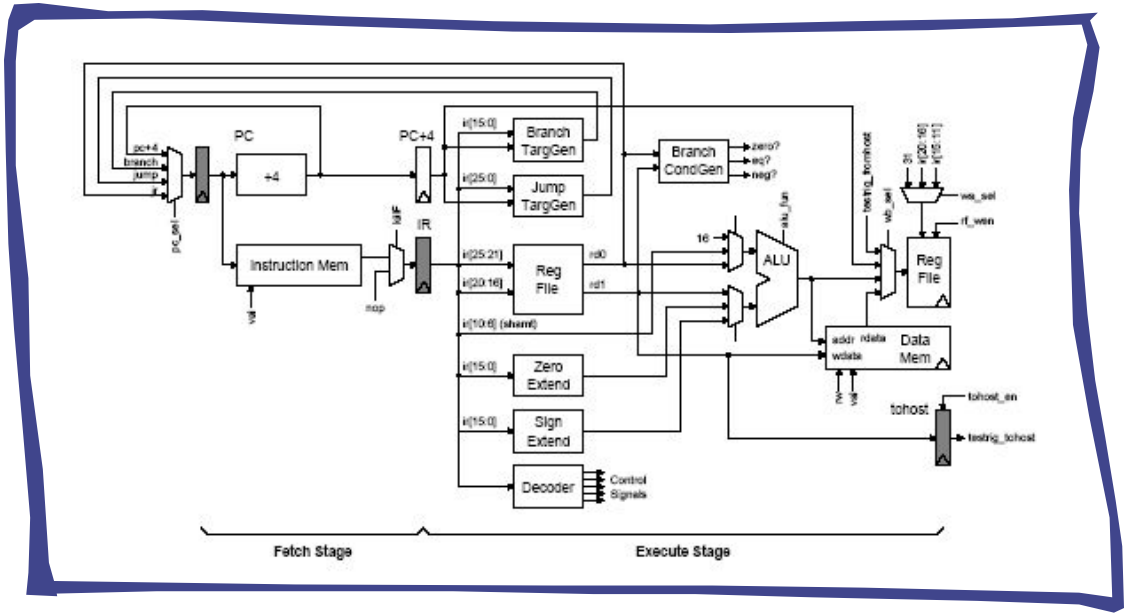
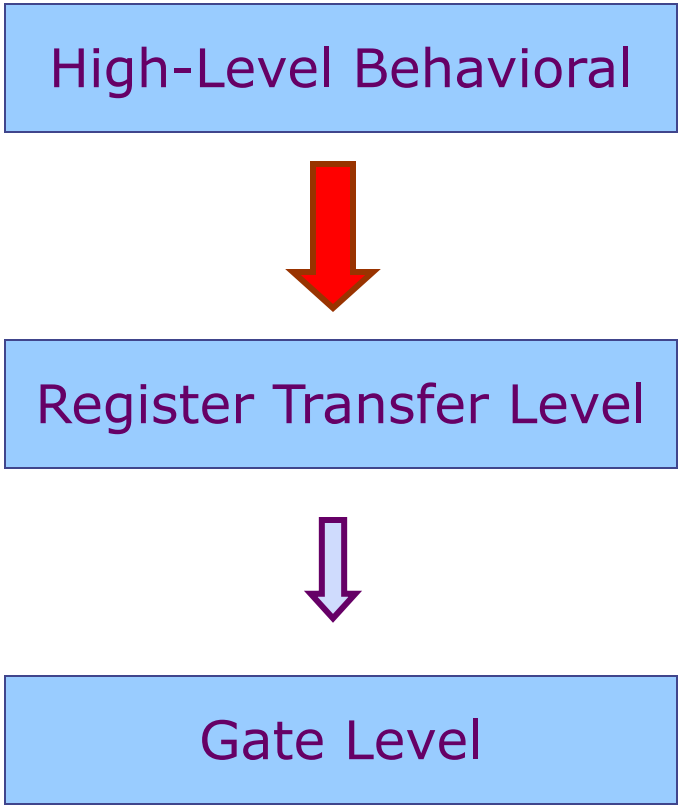
Updated January 2023

Verilog 2 - Design Examples



Modified by Michael Taylor from Arvind's MIT 6.375 slides.

Verilog can be used at several levels



A common approach is to use C/C++ for initial behavioral modeling, and for building test rigs

automatic tools to synthesize a low-level gate-level model

Writing synthesizable Verilog

Recap: Combinational logic

- Use continuous assignments (**`assign`**)

```
assign c_i = b_o + 1;
```

- Use **`always_comb`** blocks with blocking assignments (**=**)

```
always_comb
   begin
     out = 2'd0;
     if (in1 == 1)
        out = 2'd1;
     else if (in2 == 1)
        out = 2'd2;
   end
```

- Every variable should have a *default value* to avoid inadvertent introduction of latches

- Don't assign to same variable from more than one **`always_comb`** block. Race conditions in behavioral sim, synthesizes incorrectly.

Writing synthesizable Verilog

Recap: Combinational logic

- Use continuous assignments (**`assign`**)

```
assign c_i = b_o + 1;
```

- Use **`always_comb`** blocks with blocking assignments (**`=`**)

```
always_comb
   begin
      out = 2'd0;
      if (in1 == 1)
         out = 2'd1;
      else if (in2 == 1)
         out = 2'd2;
   end
```

default value

- Every variable should have a *default value* to avoid inadvertent introduction of latches

- Don't assign to same variable from more than one **`always_comb`** block. Race conditions in behavioral sim, synthesizes incorrectly.

Writing synthesizable Verilog

Recap: Combinational logic

- Use continuous assignments (**assign**)

```
assign c_i = b_o + 1;
```

- Use **always_comb** blocks with blocking assignments (**=**)

```
always_comb
   begin
      out = 2'd0;
      if (in1 == 1)
         out = 2'd1;
      else if (in2 == 1)
         out = 2'd2;
   end
```

`always` blocks allow more expressive control structures, though not all will synthesize

default value

- Every variable should have a *default value* to avoid inadvertent introduction of latches

- Don't assign to same variable from more than one **always_comb** block. Race conditions in behavioral sim, synthesizes incorrectly.

Writing synthesizable Verilog: Sequential logic

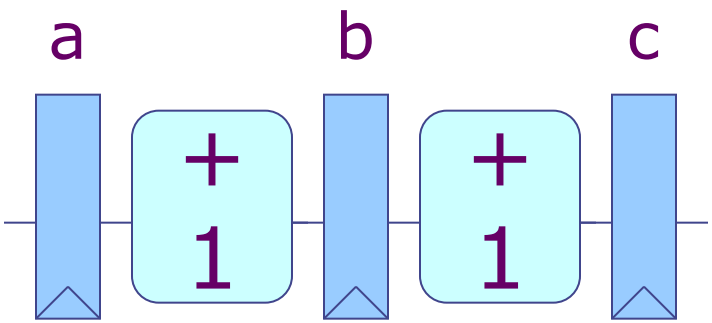
- Use **`always_ff @(posedge clk)`** only with non-blocking assignment operator (**<=**)

```
always_ff @( posedge clk )
   c_o <= c_i;
```

- Use only positive-edge triggered flip-flops for state
  - negative-edge flops should only be used in special cases
- Do not assign the same variable from more than one always_ff block. Race condition in behavioral simulation; synthesizes incorrectly.
- Do not mix blocking and non-blocking assignments
  - only use non-blocking assignments (<=) for sequential logic.
  - only use block assignments (=) for combinational logic.
- Like in software engineering, express your design as a module hierarchy that corresponds to logical boundaries in the design. Also, separate datapath and control (more later).

An Example: Good Style

```
logic a_n, b_n, c_n;
logic a_r, b_r, c_r;

always_ff @( posedge clk )
begin
  a_r <= a_n;
  b_r <= b_n;
  c_r <= c_n;
end

assign b_n = a_r + 1;
assign c_n = b_r + 1;
```

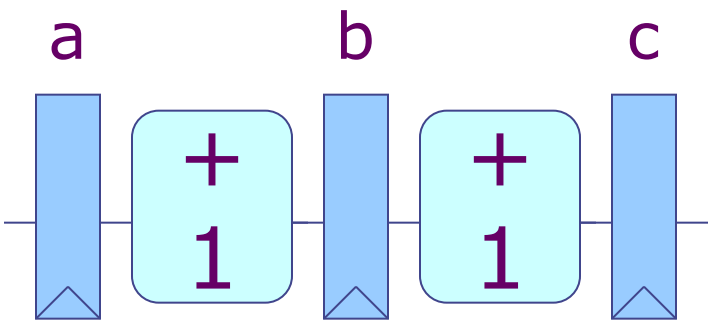


Readable, combinational and sequential logic are separated.

Standard naming: A_r is the output of the register and A_n (or A_n) is the input.

An Example: Good Style

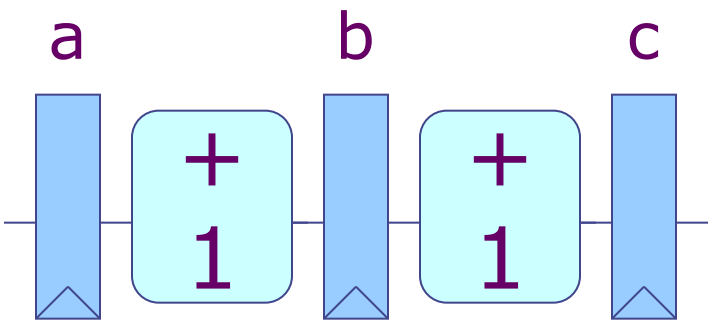
```
logic a_n;
logic b_n, c_n;
logic  a_r, b_r, c_r;

always_ff @( posedge clk )
begin
  a_r <= a_n;
  b_r <= b_n;
  c_r <= c_n;
end

always_comb
begin
  b_n = a_r + 1;
  c_n = b_r + 1;
end
```



Readable, combinational and sequential logic are separated.

Alternate implementation?

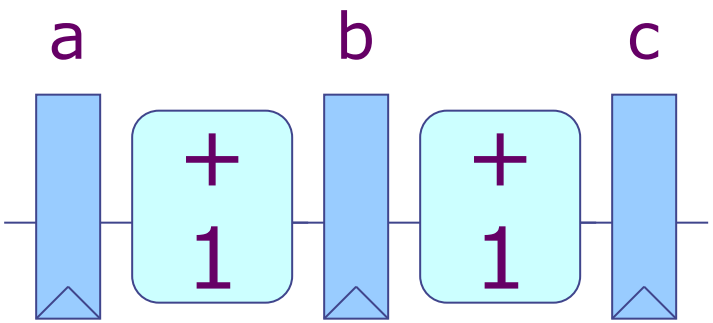
```
logic a_n, b_n, c_n;
logic a_r, b_r, c_r;

always_ff @( posedge clk )
begin
  a_r <= a_n;
  b_r <= b_n;
  c_r <= c_n;
  assign b_n = a_r + 1;
  assign c_n = b_r + 1;
end
```



Syntactically Incorrect.

An Example: Okay, but less readable?

```
logic a_n;
logic a_r, b_r, c_r;

always_ff @( posedge clk )
begin
  a_r <= a_n;
  b_r <= a_r + 1;
  c_r <= b_r + 1;
end
```



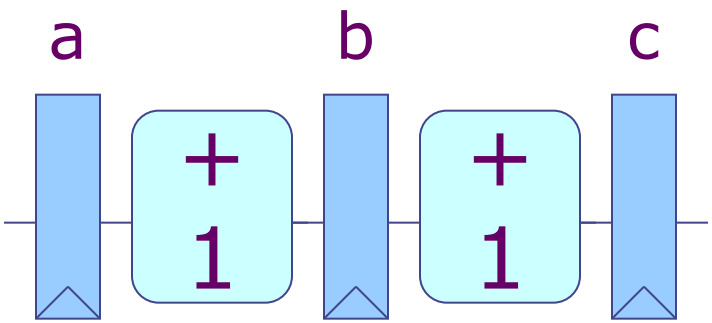
Is (b_r == a_n+1) ?

Nope - Why?

a_r <= a_n creates a register between a_n and a_r, not a wire.

Another style – multiple `always` blocks

```
logic a_n, b_n, c_n;
logic a_r, b_r, c_r;

always_ff @( posedge clk )
  a_r <= a_n;

assign b_n = a_r + 1;

always_ff @( posedge clk )
  b_r <= b_n;

assign c_n = b_r + 1;

always_ff @( posedge clk )
  c_r <= c_n;
```



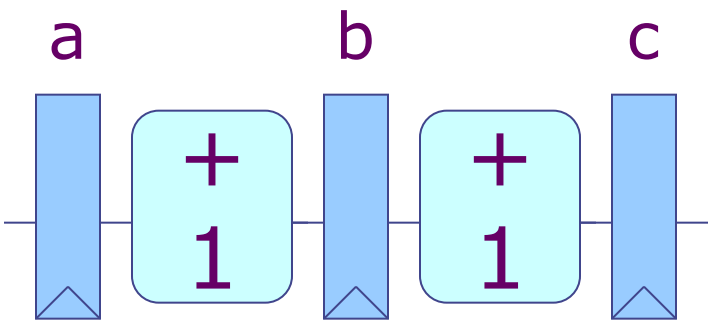
Does it have the same functionality?

Yes. But why?

It generates the same underlying circuit.

How about this one?

```
logic a_n, b_n, c_n;
logic  a_r, b_r, c_r;

always @( posedge clk )
begin
  a_r = a_n;
  b_r = b_n;
  c_r = c_n;
end

assign b_n = a_r + 1;
assign c_n = b_r + 1;
```



Will this synthesize?

→ Maybe

Is it correct?

→ No; Don't use "blocking assignments" in `@posedge clk` blocks. It creates race conditions. Also, use `always_ff` instead.

What does this do? (*This is correct but bad code.*)

```
logic b_i, c_i;
logic a_r;
logic sel;

always @( posedge clk )
begin
  a_r <= 1'b0;
  a_r <= b_i;

  if (sel)
    a_r <= c_i;
end
```

Desugar into separate comb. and seq. logic.

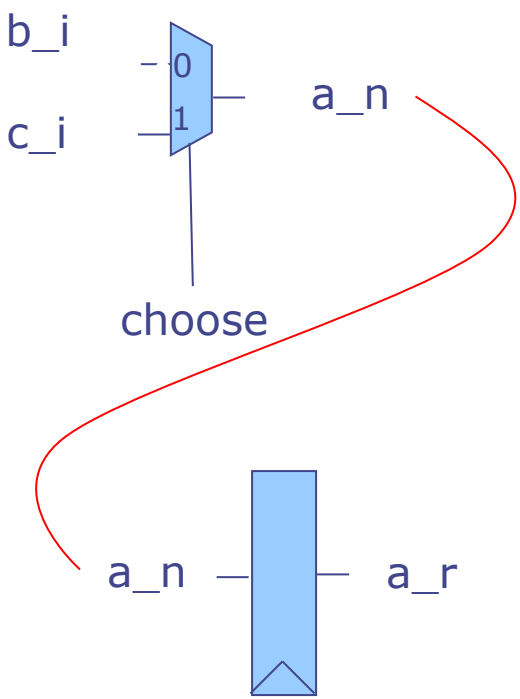
```
logic b_i, c_i;
logic a_r;
logic sel;

always @( posedge clk )
begin
  a_r <= 1'b0; // redundant!
  a_r <= b_i;

  if (sel)
    a_r <= c_i;
end
```

```
logic a_n, b_i, c_i;
logic a_r;
logic sel;

always_comb
begin
  a_n = a_r; // default;
             // rdt. but safe
  a_n = b_i;

  if (sel)
    a_n = c_i;
end

always_ff @( posedge clk )
  begin
    a_r <= a_n;
  end
```

What does this do?

For each always_comb, assign, always_ff statement, draw the gates and wires.

```
logic a_n, b_i, c_i;
logic a_r;
logic choose;

always_comb
begin
  a_n = a_r; // default

  a_n = b_i;

  if (choose)
    a_n = c_i;
end

always_ff @( posedge clk )
  begin
    a_r <= a_n;
  end
```



Verilog execution semantics

- Confusing
- Best solution is to write synthesizable verilog that corresponds exactly to logic you have *already* designed on paper. Separate combinational from sequential logic.
- Debugging is **very** difficult for Verilog. Don't write code and "see if it works." Test each "unknown" thing individually until you know what it does; then combine into larger entities.
- Before you try to simulate, manually check every wire to make sure that it is correctly (1) defined, connected to (2) source and (3) destination, and that (4) the logic driving it appears to be correct.
  - This is *way faster* than finding the same bugs in the waveform viewer!

SystemVerilog `struct` example

```
typedef struct packed {
  logic [17-1:0] instr;
  logic [10-1:0] addr;
}  instr_packet_s;

instr_packet_s ip_n, ip_A_r, ip_B_r, ip_C_r;

assign ip_n = `{addr: addr_i
               , instr: instr_i};

assign { addr_o, instr_o }
   = { ip_C_r.addr, ip_C_r.instr };


always_ff @( posedge clk )
  begin
    { ip_A_r, ip_B_r, ip_C_r } <=
           { ip_n, ip_A_r, ip_B_r };
  end
```

Good for routing bundles of identical wires together

Helpful SV Primitives

$bits( ) - number of bits required to hold value:

```
logic[31:0] foo;   // $bits(foo) = 32
struct happy foo; // $bits(struct happy) =
```

$clog2( ) - number of address bits for a memory of size
- ceiling of log base 2 of X (eg for RF impl.)

```
// $clog2(1) = 0, $clog2(2) = 1,
// $clog2(3,4) = 2, $clog2(5,6,7,8) = 3, …
```

'1,'0,'X,'Z – all 1's, all 0's, all X's, etc.

```
logic [63:0] word;  logic [3:0] byte;
  word[byte*8 +: 8] ; start at byte*8; grab 8 bits upwards
  word[byte*8 –: 8]; start at byte*8; grab 8 bits downwards
```

Enums & State Machines

```
typedef enum [2:0] { eFetch, eDecode, eExecute, eMemory, eWriteback } state_e;

state_e substate_r, substate_next;

always_ff @(posedge clk)
   substate_r <= reset ? eFetch : substate_next;

always_comb
unique case (substate_r)
      eFetch:     substate_next = eDecode;
      eDecode:  substate_next = eExecute;
      eExecute:
                        unique casez (instr_reg)
                            `SW: `LW: substate_next = eMemory;
                            default:      substate_next = eWriteback;
                        endcase
     eMemory:
                        unique casez(instruction)
                            `LW:        substate_next = eWriteback;
                             default:   substate_next = eFetch;
                        endcase
     eWriteback: substate_next = eFetch;
     default:       substate_next = eFetch;
  endcase
```

See future slide for casez usage.

Nice implementation of a state machine.

Reductions and Exponents

module foo #(parameter lg_depth_p) (....)

logic storage [(**2**lg_depth_p**)-1:0];

*2** is exponentiation, but you only use it for parameters that are determined at elaboration time. You can use $clog2 instead of passing in the log value, but sometimes you want to force the user to specific a size that is a power of two.*

Reductions

(**|** c) // all of the bits of C or'd together ("or reduce")
(**&** c) // all of the bits of C and'd together ("and reduce")
(**^** c) // all of the bits of C xor'd together ("xor reduce")

Array Signals

It is often useful to represent arrays of values; here is how you do it

```
module bsg_mux
#(parameter width_p="inv"
   , els_p=1
   , harden_p = 0
  , lg_els_lp=`BSG_SAFE_CLOG2(els_p)
  )
( input [els_p-1:0][width_p-1:0] data_i
  ,input [lg_els_lp-1:0] sel_i
  ,output [width_p-1:0] data_o
);

assign data_o = data_i[sel_i];

endmodule
```

Handle els_p=1 corner case
$clog2(1)=0 → zero width signal!
Instead $BSG_SAFE_CLOG2(1)=1

"Generate" if/else Statements

```
module add_or_subtract
  #(parameter add_not_sub_p="inv")
 ( input  [3:0] a_i
   , input  [3:0] b_i
   , output [3:0] result_o );

if (add_not_sub_p)
  begin: add
       assign result _o = a_i + b_i;
  end
else
  begin: sub
       assign result _o = a_i - b_i;
  end

endmodule
```

Constant parameter at synthesis time i.e., when verilog is turned into gates

arbitrary label

generates an adder or a subtractor, not an add/subtractor!

Contrast to an if/else statement inside an always block that logic is making a run-time decision.

“generate” for loops

```
module bsg_dff_gatestack
#(width_p="inv",harden=1)
(input          [width_p-1:0] i
,input      [width_p-1:0] clk
,output logic[width_p-1:0] o
);

genvar j;
for (j=0;j < width_p; j=j+1)
    begin: rof
        always_ff @(posedge clk[j])
            o[j] <= i[j];
        end
endmodule
```

at synthesis time
replicate logic
width_p times

arbitrary label->appears in output verilog

You can put
either RTL
or structural
Verilog here.

Generate Statements: For

Stamp out hardware with for loops!

```
module bsg_crossbar_o_by_i
#( parameter i_els_p = -1
   ,parameter o_els_p = -1
   ,parameter width_p = -1 )
 ( input [i_els_p-1:0][width_p-1:0] i
  ,input [o_els_p-1:0][i_els_p-1:0] sel_oi_one_hot_i
  ,output [o_els_p-1:0][width_p-1:0] o
 );

genvar lineout;

for(lineout=0; lineout<o_els_p; lineout++)
begin
  bsg_mux_one_hot #( .width_p(width_p) ,.els_p (i_els_p) )
        mux_one_hot ( .data_i (i)
                      ,.sel_one_hot_i(sel_oi_one_hot_i[lineout])
                      ,.data_o (o[lineout])
    );
end
endmodule // bsg_crossbar_o_by_i
```

Generate Statements: IF and FOR together!

Stamp out hw conditionally!

add particular elements of vector; determined statically

```
module bsg_selective_sum
#( parameter i_els_p = -1
   ,parameter width_p = -1
   , parameter add_val_p = -1)
 ( input [i_els_p-1:0][width_p-1:0] i
  ,output width_p-1:0] o );

genvar index;
logic [i_els_p:0][width_p-1:0] val;
assign val[0] = width_p ' 0;  assign o = val[i_els_p];

for(index=0; index<i_els_p; index=index+1)
begin : rof
  if (add_val_p[index])
   begin: add
     assign val[index+1] = val[index] + i[index];
   end // add
  else
  begin: no_add
    assign val[index+1] = val[index];
  end  // no_add
end // rof
endmodule
```

*Always label **if** or **for** statements*

Pattern Matching

```
`define LW    32'b100011_?????_?????_?????_?????_??????
`define SW    32'b101011_?????_?????_?????_?????_??????
`define ADDIU 32'b001001_?????_?????_?????_?????_??????
`define BNE   32'b000101_?????_?????_?????_?????_??????
```

```
reg writes_rf_c;  // aka RegWrite;

always_comb
  unique casez (IR)
      `LW:    writes_rf_c = 1'b1;
      `SW:    writes_rf_c = 1'b0;
      `BNE:   writes_rf_c = 1'b0;
      `ADDIU: writes_rf_c = 1'b1;
      …
      default: writes_rf_c = 1'b0;
  endcase
```

```
reg writes_dmem_c; // aka MemWrite;

always_comb
  unique casez (IR)
      `SW:    writes_dmem_c = 1'b1;
      …
      default: writes_dmem_c = 1b'0;
  endcase
```

Useful Discipline: Always give a **`default:`** *for* ***`unique casez`****, even if it is empty.*

(This enables pass-thru of settings prior to the casez statement; otherwise synthesizer will assume it's don't care.)